Shortest Path

Bellman–Ford algorithm

Shortest path

- **การหาเส้นทางที่สั้นที่สุด**ระหว่างโหนดสองโหนดในกราฟเป็นปัญหาสำคัญที่มี application ในชีวิตประจำวันใช้มากมาย ตัวอย่างเช่นปัญหาที่เกี่ยวกับ เครือข่ายถนนจะคำนวณความยาวของระยะทางสั้นที่สุดระหว่างเมืองสองเมืองเมื่อกำหนดความยาวของถนนมาให้
- ในกราฟแบบไม่มีน้ำหนัก แก้ปัญหานี้อย่างไรดี
- ความยาวของเส้นทาง**เท่ากับจำนวน**ของเส้นเชื่อม
- ดังนั้นเราจึงสามารถใช้ breadth-first search ในการหา shortest path

Shortest path

- ดังนั้นเราจะสนใจกราฟแบบมีน้ำหนัก
- เราจะมาดูอัลกอริทึมในการหา shortest paths 3 แบบ
  - Bellman-Ford algorithm
  - Dijkstra's algorithm
  - Floyd-Warshall algorithm

Bellman–Ford algorithm

- **Bellman–Ford algorithm** หา shortest paths จาก**โหนดเริ่มต้น**ไปยัง**ทุกโหนด**ในกราฟ

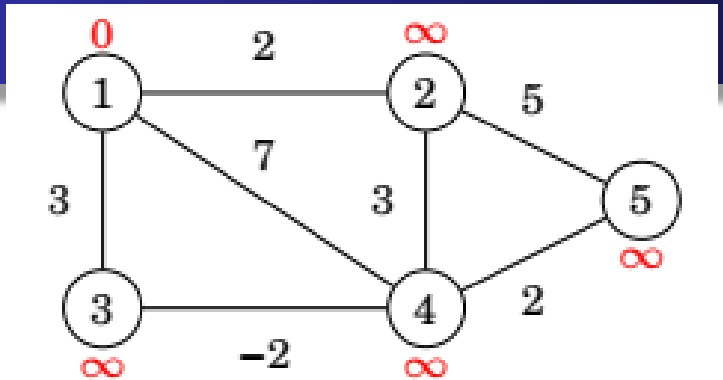
- อัลกอริทึมสามารถประมวลผลกราฟได้ทุกรูปแบบ ทั้งนี้กราฟต้อง**ไม่มี** cycle ที่มีความยาวเป็นลบ (cycle with negative length หรือ negative cycle)

- แต่!!! ถ้ากราฟ**มี** **negative cycle** อัลกอริทึมนี้สามารถ**ตรวจสอบได้**

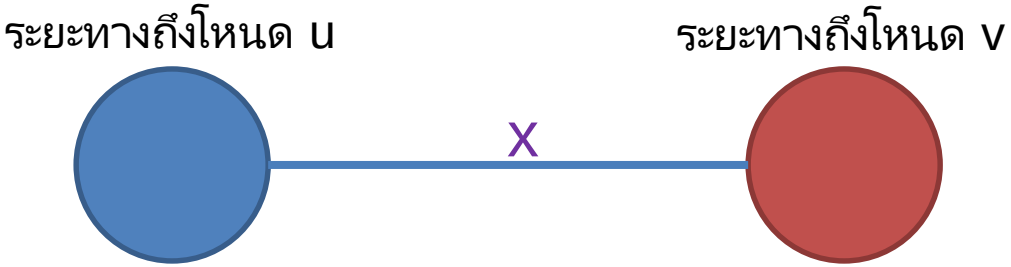
Bellman-Ford algorithm

- อัลกอริทึมจะเก็บระยะทางจากโหนดเริ่มต้นไปยังทุกโหนดในกราฟเริ่มต้น
  - ระยะทางจากโหนดเริ่มต้นเป็น 0
  - และระยะทางไปยังทุกโหนดเป็น infinity

- การทำงานจะทำเป็น**รอบ**
- ในแต่ละรอบอัลกอริทึม พิจารณา**ทุกเส้นเชื่อม**
- แล้วพยายาม**ลด**ระยะทาง ทำจนกระทั่งไม่สามารถลดระยะทางได้อีก

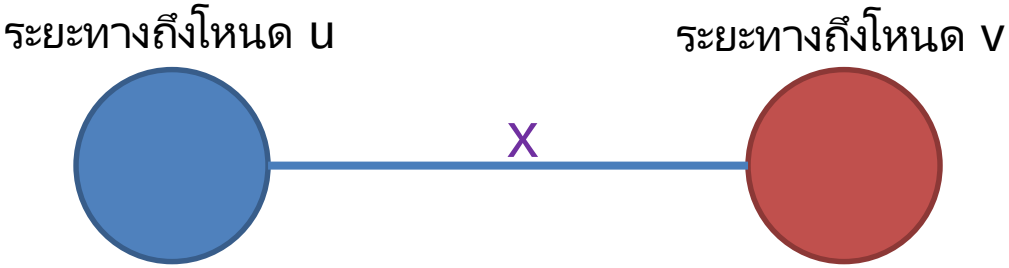


- แต่ละโหนดในกราฟจะถูกกำหนดระยะทาง เริ่มต้นระยะทางของ start node(1) เป็น 0 และระยะทางไปยังโหนดอื่นๆ ทุกโหนดเป็น infinity
- ในแต่ละรอบอัลกอริทึมจะ**พิจารณาทุกเส้นเชื่อม**เพื่อค้นหาเส้นเชื่อมที่ลดระยะทางได้



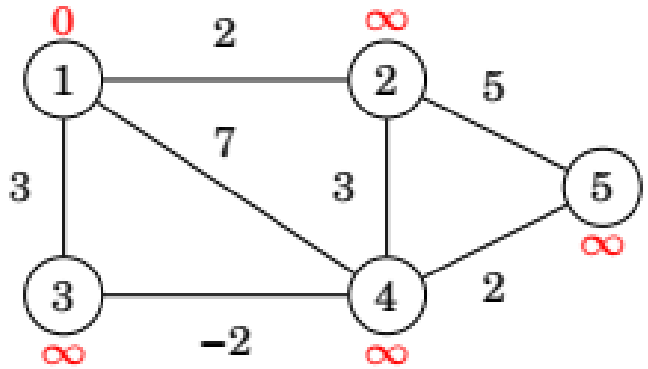
ระยะทางถึงโหนด v = ระยะทางถึงโหนด u + x

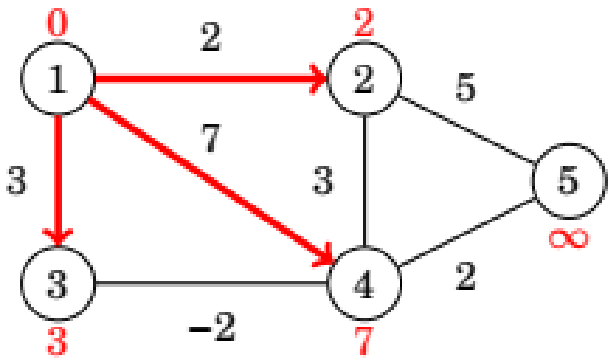
- รอบแรก พิจารณาทุกเส้นเชื่อม
- พบว่าเส้นเชื่อมจากโหนด 1 ลดระยะทางได้



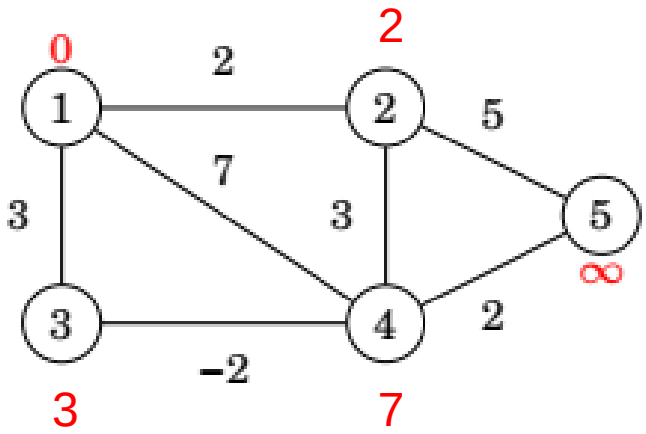
ระยะทางถึงโหนด v = ระยะทางถึงโหนด u + x

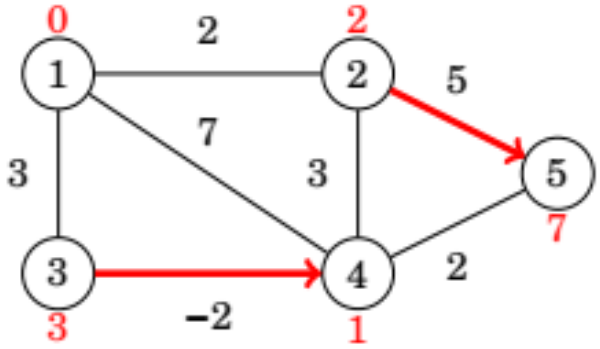
- อย่างน้อยจะลดระยะทางของโหนดที่ติดกับโหนด 1 ได้



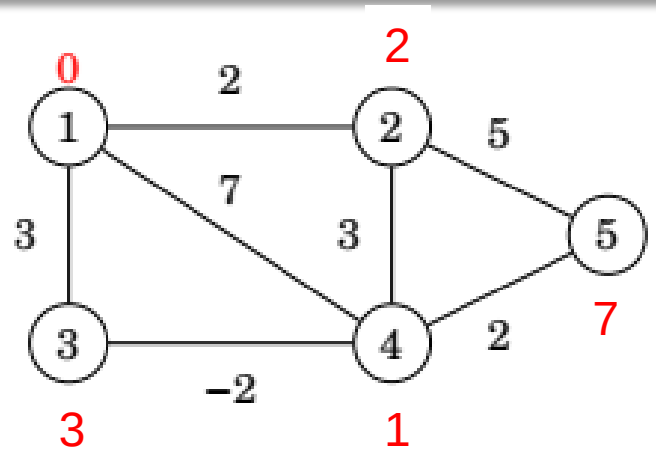


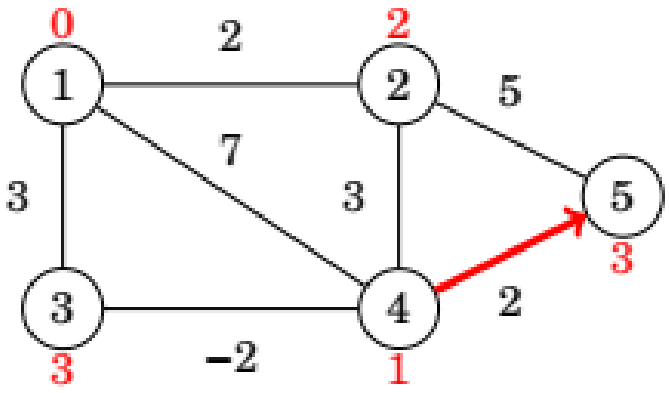
- หลังจากนั้นรอบต่อมา พิจารณา**ทุกเส้น**เชื่อมอีก
- พบว่าอย่างน้อย เส้นเชื่อม 2 –> 5 และ 3 –> 4 จะลดระยะทาง





- แล้วมีการปรับอีก

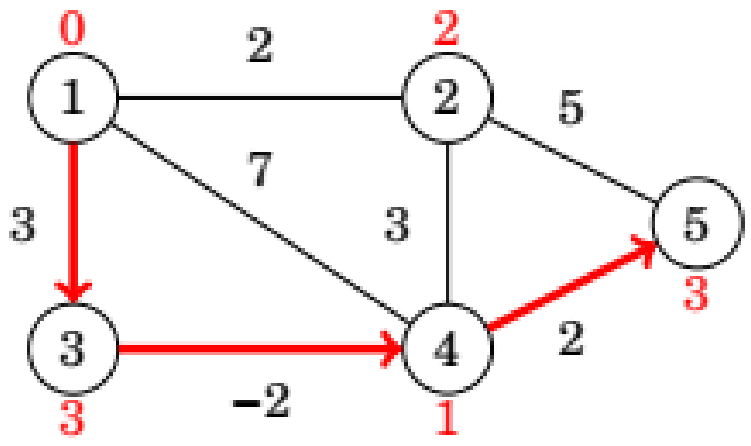




- หลังจากนี้ ไม่มีเส้นเชื่อมที่สามารถลดระยะทางได้

- นั่นหมายความว่านี่เป็นระยะทางสุดท้าย และเราได้คำนวณระยะทางสั้นที่สุดจากโหนดเริ่มต้นไปยังทุกโหนดแล้ว

- **ตัวอย่าง** ระยะทางที่สั้นที่สุดจากโหนด 1 ไปโหนด 5 มีค่าเป็น 3 ซึ่งสอดคล้องกับเส้นทางที่ได้



Implementation

- การ implement ของ Bellman–Ford algorithm จะตัดสินระยะทางที่สั้นที่สุดจากโหนด x ไปยังทุกโหนดในกราฟ
- เนื่องจากในแต่ละรอบ เราพิจารณาทุกเส้นเชื่อม การเก็บกราฟแบบใดทำให้เราใช้งานต่อง่ายในกรณีนี้
- Edge list
- สมมติว่ากราฟถูกเก็บด้วยวิธี Edge list

Implementation

- สมมติว่ากราฟถูกเก็บด้วยวิธี **Edge list**
- edges ประกอบด้วย **tuples** ในรูปแบบ **(a, b, w),** หมายความว่ามีเส้นเชื่อมจาก a ไป b ด้วยน้ำหนัก w
- อัลกอริทึมใช้เวลาทำงาน **n–1** รอบ ในแต่ละรอบนั้นจะพิจารณาทุกเส้นเชื่อมและพยายามลดระยะทางจาก x ไปยังทุกโหนดในกราฟ
- ค่าคงที่ INF แทนระยะทางเป็น infinity

```
for (int i = 1; i <= n; i++)
  distance[i] = INF;

distance[x] = 0;

for (int i = 1; i <= n-1; i++) {
  for (auto e : edges) {
    int a, b, w;
    tie(a, b, w) = e;
    distance[b] = min(distance[b], distance[a]+w);
  }
}
```

วิเคราะห์

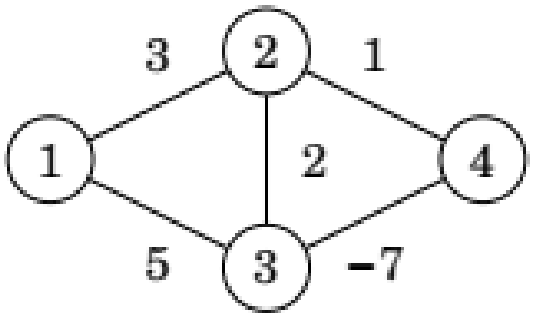
- เวลาในการทำงานของ algorithm เป็น **O(nm)** เพราะว่าอัลกอริทึมทำงาน **n–1** รอบและแต่ละรอบตรวจสอบทุกเส้นเชื่อม **m** เส้น

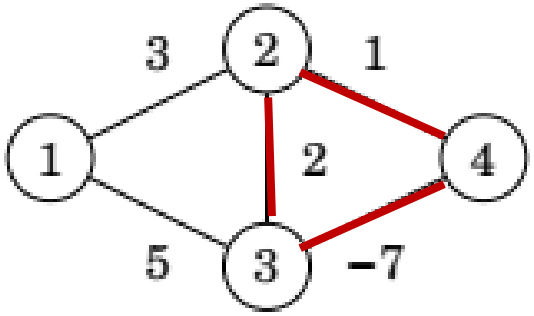
- ถ้าไม่มี negative cycles ในกราฟ ทุกระยะทางจะเสร็จสิ้นหลังจาก n–1 รอบเพราะว่า shortest path แต่ละเส้นมีเส้นเชื่อมได้ไม่เกิน n–1 เส้น

- ในทางปฏิบัติแล้วระยะทางสุดท้ายมักจะเสร็จก่อน n–1 รอบ
- ดังนั้นสามารถเขียน code ดักว่าถ้า**ไม่มีอะไรเปลี่ยน**แล้วหยุดการทำงานก่อนได้เพื่อลดจำนวนรอบในการทำงาน

การจัดการกับ Negative cycle

- Bellman–Ford algorithm สามารถใช้ตรวจสอบว่ากราฟมี cycle ที่มีความยาวเป็นลบได้ ตัวอย่างเช่นกราฟที่มี negative cycle





- มี negative cycle 2 –> 3 –> 4 –> 2 เป็น cycle ที่มีความยาวเป็น –4
- ถ้ากราฟมี negative cycle เราสามารถทำให้**ระยะทางสั้นลงได้เรื่อยๆ**
- ดังนั้น shortest path จะไม่มีความหมายในเหตุการณ์นี้

- ถ้ากราฟมี negative cycle เราสามารถทำให้**ระยะทางสั้นลงได้เรื่อยๆ**
- โดยปกติ อัลกอริทึมทำงานมันต้องเสร็จภายใน กี่รอบ
- N–1 รอบ
- ถ้ารอบที่ N ยังลดได้อีก หมายความว่าอะไร

- Negative cycle สามารถถูกตรวจสอบได้จาก Bellman–Ford algorithm โดยการรันอัลกอริทึม *N* รอบ
- ถ้ารอบสุดท้ายระยะทางลดลงได้อีกแสดงว่ากราฟมี negative cycle
- สังเกตว่าอัลกอริทึมนี้สามารถถูกใช้เพื่อหา negative cycle โดยไม่สนใจว่าโหนดเริ่มต้นเป็นโหนดใดได้
- ก่อนหน้าที่แสดงแบบ Edge list ต่อไปแสดงแบบ Adjacency list ให้ดู

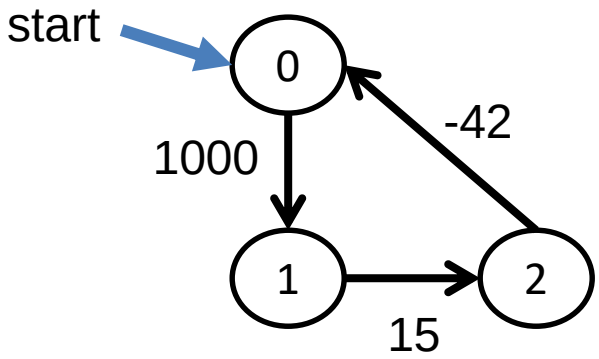
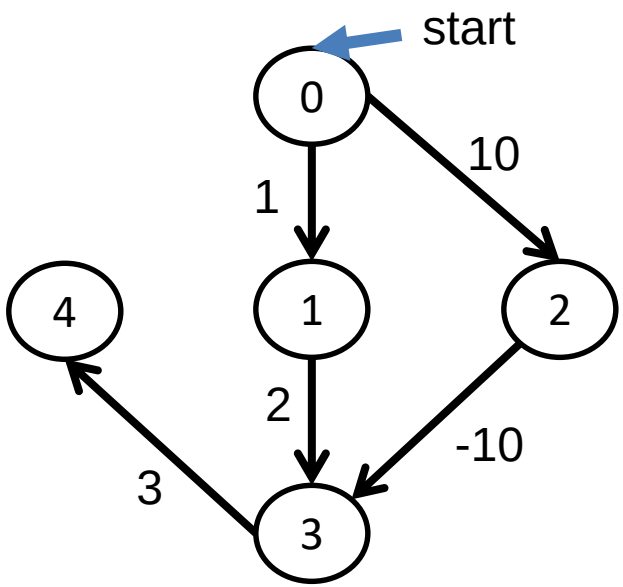
```
#include <bits/stdc++.h>
using namespace std;

typedef pair<int, int> ii;
typedef vector<int> vi;
typedef vector<ii> vii;
#define INF 1000000000

int main() {
  int V, E, s, u, v, w;
  vector<vii> AdjList;
  scanf("%d %d %d", &V, &E, &s);
  AdjList.assign(V, vii());
  for (int i = 0; i < E; i++) {
    scanf("%d %d %d", &u, &v, &w);
    AdjList[u].push_back(ii(v, w));
  }
```

```
vi dist(V, INF); dist[s] = 0;
for (int i = 0; i < V - 1; i++)  // ทำ V-1 รอบ
  for (int u = 0; u < V; u++)    // สอง loop นี้ คือพิจารณาทุกเส้นเชื่อม
    for (int j = 0; j < (int)AdjList[u].size(); j++) {
      ii v = AdjList[u][j];
      dist[v.first] = min(dist[v.first], dist[u] + v.second); // ลดความยาว
    }
bool hasNegativeCycle = false;
for (int u = 0; u < V; u++)      // ทำอีกรอบเพื่อเช็ค
  for (int j = 0; j < (int)AdjList[u].size(); j++) {
    ii v = AdjList[u][j];
    if (dist[v.first] > dist[u] + v.second) // ถ้าไม่มีจะเป็น false
      hasNegativeCycle = true;      // ถ้ามีแสดงว่ามี negative cycle
  }
printf("Negative Cycle Exist? %s\n", hasNegativeCycle ? "Yes" : "No");

if (!hasNegativeCycle)
  for (int i = 0; i < V; i++)
    printf("SSSP(%d, %d) = %d\n", s, i, dist[i]);
return 0;
}
```

แบบ Adjlist https://bit.ly/2WmlAjF

- ทดลองกราฟนี้